PRVA LAB VEZBA: Korišćenjem programskog jezika Python, napisati funkciju uredi, koja svaki od prvih N elemenata uvecava za definisanu vrednost a preostale umanjuje za definisanu vrednost. Funkciji se prosleđuje lista koja sadrži samo numeričke vrednosti i vrednost koja treba da se uvecava, odnosno umanjuje. Primer: uredi([1, 2, 3, 4, 5], 3, 1) = [2, 3, 4, 3, 4]

In [1]:

```
def uredi(lista, N, vrednost):
    for i in range(0, N):
        lista[i]+=vrednost
    for i in range(N, len(lista)):
        lista[i]-=vrednost
    return lista

print(uredi([1,2,3,4,5],3,1))
```

```
[2, 3, 4, 3, 4]
```

DRUGA LAB VEZBA: Korišćenjem programskog jezika Python, napisati funkciju suma, koja prvenstveno određuje proizvod elemenata u svakoj podlisti unutar prosleđene liste, a zatim sumu tako dobijenih elemenata. Zabranjeno je korišćenje petlji (osim u comprehension sintaksi) i funkcije sum i prod.

In [4]:

```
from functools import reduce
```

In [5]:

```
def suma(lista):
    proizvodi = [
        reduce(lambda x, y: x*y, podlista, 1)
        for podlista in lista
    ]
    return reduce(lambda x, y: x+y, proizvodi)
print(suma([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
```

```
630
```

TRECA LAB VEZBA: 14. Napisati funkciju koja određuje da li je usmereni graf jako povezan graf (da li postoji put između bilo koja dva čvora u grafu).

In [3]:

```
#da li je usmeren graf jako povezan? da li za svaka dva cvora

def dfs(graf, start, poseceni): #algoritam trazenja po dubini
    poseceni.add(start) #posetili smo trenutni cvor

    for sused in graf[start]: #prolazimo kroz sve cvorove do
        if sused not in poseceni: #ako cvor nije posecen obil
            dfs(graf, sused, poseceni) #dfs ide sto dublje mo


def obrni_graf(graf): #ako u originalu postoji u->v, u obrnut
    obrnuti = {v: [] for v in graf} #prazna linija susedstva

    for u in graf:
        for v in graf[u]: #prolazimo kroz sve grane u->v
            obrnuti[v].append(u) #okrecemo smer grane u v->u

    return obrnuti


def jako_povezan(graf):
    start = next(iter(graf)) #krecemo od bilo kog cvora u gra

    poseceni = set()
    dfs(graf, start, poseceni)

    if len(poseceni) != len(graf): #ako postoji makar 1 cvor
        return False

    obrnuti = obrni_graf(graf) #da li svi cvorovi mogu da doc
    poseceni = set()
    dfs(obrnuti, start, poseceni)

    if len(poseceni) != len(graf):
        return False
```

```
    return True #ako su oba uslova ispunjena graf jeste jako
```

In [4]:

```
graf1 = {
    0: [1],
    1: [2],
    2: [0]
}

print(jako_povezan(graf1))
```

```
True
```

In [2]:

```
graf2 = {
    0: [1],
    1: [2],
    2: []
}

print(jako_povezan(graf2))
```

```
False
```

Nacin racunske vezbe

In [13]:

```
import queue

def depth_first_search(graph, start, visited): #ne idemo od p
#iz jednog čvora može da otkrije sve čvorove do kojih postoji
    stack_nodes = queue.LifoQueue(len(graph))
    stack_nodes.put(start) #start je posecen
    visited.add(start) #dodajemo start na stek

    while not stack_nodes.empty(): #dokle god stek nije prazc
        node = stack_nodes.get()

        for dest in reversed(graph[node]): #reversed se koris
            if dest not in visited:
                visited.add(dest)
                stack_nodes.put(dest) #obilazimo graf i u vis

def reverse(graph): #usmereni graf!
    r_graph = {} #pravimo prazan graf

    for node in graph:
        r_graph[node] = [] #inicijalizacija cvorova

#reversed_graph = {
#    0: [],
#    1: [],
#    2: []
#}

    for node in graph: #ako postoji node->dest , dobijamo des
        for dest in graph[node]:
            r_graph[dest].append(node)

    return r_graph #vracamo NOVI graf

def check_graph(graph):
    start = next(iter(graph)) #uzimamo bilo koj cvor

    visited = set()#cuva jedinstvene vrednosti, redosled nije
    depth_first_search(graph, start, visited) #dfs obilazi sv
```

```
    if len(visited) != len(graph): #ako neki cvor nije obidje
        #len graph je broj kljuceva u recniku
        return False

    r_graph = reverse(graph)
    visited.clear()
    depth_first_search(r_graph, start, visited)

    return len(visited) == len(graph) #ako su svi cvorovi pos
```

In [14]:

```
graph = {
    0: [1],
    1: [2],
    2: [0]
}

print(check_graph(graph))
```

```
True
```

In [15]:

```
graph2 = {
    0: [1],
    1: [2],
    2: []
}

print(check_graph(graph2))
```

```
False
```

CETVRTA LAB VEZBA: Dat je skup domina u kome najveća ima dve četvorke na polovinama i ima ih ukupno 15. Igrač u svakom koraku postavlja jednu dominu na jedan od krajeva poštujući pravila igre domina.

Poređati sve domine iz ovog skupa u niz poštujući pravila igre domina

PETA LAB VEZBA: Constraint Satisfaction Problem: Implementirati Backtracking traženje u kombinaciji sa Forward Checking tehnikom i MRV (Minimum Remaining values) heuristikom za popunjavanje matrice dimenzija 4x4, brojevima od 1 do 4, tako da isti broj ne sme da se nađe u istoj vrsti, kao ni u istoj koloni (sudoku).

In [2]:

```
import sys

def is_end(graph):
    return all(graph[node]["value"] != 0 for node in graph)

def get_mrv_node(graph, visited):
    best_node = None
    min_size = float('inf')

    for node in graph:
        if node not in visited:
            domain_size = len(graph[node]["domain"])
            if domain_size < min_size:
                min_size = domain_size
                best_node = node
    return best_node

def csp(graph, visited):
    if is_end(graph):
        return True

    current_node = get_mrv_node(graph, visited)
    if current_node is None:
        return False

    current = graph[current_node]
    original_domain = list(current['domain'])

    for value in original_domain:
        visited.add(current_node)
        current['value'] = value

        saved_domains = {}
        valid = True

        for neighbor in current["adj_nodes"]:
            if graph[neighbor]["value"] == 0:
                if value in graph[neighbor]["domain"]:
                    if len(graph[neighbor]["domain"]) == 1:
                        valid = False
```

```
                        break
                    saved_domains[neighbor] = graph[neighbor]
                    graph[neighbor]["domain"].remove(value)

        if valid:
            if csp(graph, visited):
                return True

        current['value'] = 0
        for node_name, old_domain in saved_domains.items():
            graph[node_name]["domain"] = old_domain
        visited.remove(current_node)

    return False

def build_graph(size=4):
    graph = {}
    for row in range(size):
        for col in range(size):
            node = f'P{row*size + col + 1}'
            graph[node] = {"domain": set(range(1, size+1)), '

    for row in range(size):
        for col in range(size):
            node = f'P{row*size + col + 1}'
            adj = set()
            for i in range(size):
                if i != col:
                    adj.add(f'P{row*size + i + 1}')
                if i != row:
                    adj.add(f'P{i*size + col + 1}')
            graph[node]["adj_nodes"] = list(adj)
    return graph

def run_csp(graph):
    visited = set()
    csp(graph, visited)
    size = int(len(graph) ** 0.5)
    return [graph[f'P{i}']["value"] for i in range(1, size*si

graph = build_graph()
results = run_csp(graph)
```

```python
size = 4
for i, val in enumerate(results):
    sys.stdout.write(str(val) + " ")
    if (i + 1) % size == 0:
        sys.stdout.write("\n")
```

```
1 2 3 4
2 4 1 3
3 1 4 2
4 3 2 1
```

In [ ]: